Politechnika Łódzka
Wydział Elektrotechniki, Elektroniki, Informatyki i Automatyki

***Michał Szulejko***
***245751***

PRACA DYPLOMOWA
inżynierska
na kierunku Elektronika i Telekomunikacja

# Implementation of Convolutional Neural Network on FPGA for MNIST Digit Recognition

Instytut Elektroniki

**Promotor: Dr inż. Robert Strąkowski**

ŁÓDŹ 2026

**Abstract**

Neural networks were first conceived decades ago, but their potential for deep architectures remained unrealized due to insufficient computational power. The rise of modern graphics processing units (GPUs), capable of massively parallel matrix operations, was what finally made training deep networks feasible. While training deep networks demands extraordinary computational resources, inference requires significantly less — motivating a shift toward power-efficient hardware such as Field-Programmable Gate Arrays (FPGAs) and Application-Specific Integrated Circuits (ASICs) for deployment in embedded systems.

This thesis presents a complete end-to-end implementation of a LeNet-5 for Modified National Institute of Standards and Technology (MNIST) handwritten digit classification on a Basys3 FPGA board featuring the Xilinx Artix-7 XC7A35T chip. The project encompasses three main components: (1) training a LeNet-5 Convolutional Neural Network (CNN) model using PyTorch with post-training static quantization to 8-bit integers, (2) implementing the inference engine in Verilog RTL as a 44-state finite state machine (FSM) with specialized Block RAM (BRAM) latency handling, and (3) comprehensive verification through Python simulation and hardware testing via Universal Asynchronous Receiver-Transmitter (UART) communication.

Key features include a complete quantization pipeline that preserves accuracy while enabling efficient fixed-point arithmetic, a modular RTL architecture using hybrid memory (distributed RAM for activation buffers, Block RAM for weight storage), and a robust UART protocol for weight loading and real-time inference. The implementation demonstrates the viability of deploying neural networks on resource-constrained FPGAs for embedded applications.

The system achieves 97.6% classification accuracy after quantization while maintaining bit-exact correspondence between Python simulation and FPGA hardware inference.

**Keywords:** FPGA, CNN, MNIST, Quantization, Verilog

**Streszczenie**

Sieci neuronowe zostały opracowane już kilkadziesiąt lat temu, lecz ich potencjał dla głębokich architektur pozostawał niewykorzystany z powodu niewystarczającej mocy obliczeniowej. Dopiero pojawienie się nowoczesnych procesorów graficznych (GPU), zdolnych do masowo równoległych operacji macierzowych, umożliwiło efektywne trenowanie głębokich sieci. Choć trenowanie głębokich sieci wymaga ogromnych zasobów obliczeniowych, sama inferencja pochłania ich znacznie mniej — co uzasadnia stosowanie energooszczędnego sprzętu, takiego jak układy FPGA i ASIC, w systemach wbudowanych.

Niniejsza praca przedstawia kompletną implementację konwolucyjnej sieci neuronowej LeNet-5 przeznaczonej do klasyfikacji odręcznie pisanych cyfr ze zbioru MNIST. System zrealizowano na płycie rozwojowej Basys3 z układem Xilinx Artix-7 XC7A35T. Projekt obejmuje trzy główne etapy: (1) uczenie modelu LeNet-5 we frameworku PyTorch z kwantyzacją po treningu do 8-bitowych liczb całkowitych, (2) implementację silnika inferencji w języku Verilog jako 44-stanowego automatu skończonego ze specjalizowaną obsługą opóźnień pamięci BRAM, oraz (3) weryfikację poprawności działania poprzez symulację w języku Python i testy na rzeczywistym sprzęcie z wykorzystaniem komunikacji UART.

Do kluczowych osiągnięć projektu należą: kompletny proces kwantyzacji zachowujący wysoką dokładność klasyfikacji przy jednoczesnym umożliwieniu efektywnej arytmetyki stałoprzecinkowej, modularna architektura wykorzystująca hybrydową hierarchię pamięci (pamięć rozproszoną dla buforów aktywacji, pamięć blokową dla przechowywania wag sieci) oraz niezawodny protokół komunikacji UART pozwalający na ładowanie wag i przeprowadzanie inferencji w czasie rzeczywistym. System osiąga 97.6% dokładności klasyfikacji po kwantyzacji, zachowując bitową zgodność między symulacją programową w Pythonie a sprzętową implementacją na FPGA.

**Słowa kluczowe:** FPGA, CNN, MNIST, kwantyzacja, Verilog

# Contents

# Chapter 1

# Introduction

## 1.1 Background and Motivation

Deep neural networks demand extraordinary computational resources, performing billions of multiply-accumulate (MAC) operations for a single inference. Traditional CPU architectures proved inadequate for such workloads, prompting the exploration of specialized accelerators. The watershed moment arrived in 2012 when AlexNet demonstrated that Graphics Processing Units could dramatically accelerate deep learning training through massively parallel processing, achieving breakthrough performance on the ImageNet classification challenge (Krizhevsky, Sutskever, and Hinton 2012). This discovery catalyzed widespread adoption of GPUs for neural network development, fundamentally reshaping the AI research landscape and enabling the deep learning revolution.

Despite their training dominance, GPUs face significant challenges in deployment scenarios. Power consumption exceeding 200 watts, substantial physical size, and high cost make GPUs impractical for embedded and edge applications. Field-Programmable Gate Arrays have emerged as compelling alternatives for neural network inference, offering reconfigurable hardware that bridges the gap between software flexibility and hardware efficiency (Venieris and Bouganis 2016). FPGAs provide distinct advantages: energy consumption in single-digit watts, deterministic latency for real-time systems, support for custom bit-width arithmetic enabling aggressive quantization, and the ability to implement application-specific datapaths. These characteristics make FPGAs particularly suitable for resource-constrained deployments in autonomous systems, industrial automation, telecommunications infrastructure, and aerospace applications where power efficiency and reliability are paramount.

## 1.2 Scope of the Thesis

This work demonstrates a complete implementation of the classic LeNet-5 convolutional neural network for MNIST digit classification on the Xilinx Artix-7 XC7A35T FPGA (Digilent Basys3 board). The system integrates three components: PyTorch-based train-

ing with post-training quantization to 8-bit integers, Verilog RTL inference engine implemented as a 44-state finite state machine with dedicated BRAM latency handling, and comprehensive verification ensuring bit-exact correspondence between software simulation and hardware execution. The LeNet-5 architecture processes 28×28 grayscale images through two convolutional layers (with 5×5 kernels), average pooling, and three fully-connected classification layers, with all weights and activations quantized to enable efficient fixed-point arithmetic on FPGA fabric. (LeCun, Bottou, et al. 1998) Implementation highlights include a complete quantization workflow that accounts for scale propagation through average pooling layers while preserving classification accuracy, a modular RTL design utilizing distributed RAM for convolutional layers and Block RAM for fully-connected weights, a 256-entry tanh lookup table for activation functions, and a robust UART-based protocol for weight loading and real-time inference control. The implementation achieves 97.6% accuracy on MNIST classification (bit-exact match with quantized Python model) and matches floating-point PyTorch accuracy with zero degradation. These results validate FPGA-based neural network deployment as viable for embedded applications where power efficiency and deterministic latency are critical requirements.

## 1.3 Historical Context

LeNet-5 is a pioneering convolutional neural network architecture developed by Yann LeCun and colleagues in the 1990s for handwritten digit recognition. This architecture established several fundamental principles that remain relevant in modern deep learning:

- **Local Receptive Fields:** small convolutional kernels that detect local features,
- **Weight Sharing:** the same kernel applied across the entire input, reducing parameters,
- **Subsampling (Pooling):** spatial dimension reduction for translation invariance,
- **Hierarchical Feature Extraction:** progressive abstraction from edges to digits.

What distinguishes LeNet-5 from classical machine learning models such as decision trees, random forests, and support vector machines is its ability to automatically learn hierarchical feature representations directly from raw pixel data. Traditional machine learning approaches require manual feature engineering—extracting handcrafted features like edge histograms, Histogram of Oriented Gradients (HOG), or Scale-Invariant Feature Transform (SIFT) descriptors—before classification can occur. LeNet-5 eliminates this bottleneck through end-to-end learning, where convolutional layers progressively discover optimal features during training, from low-level edges to high-level digit shapes. This

architectural advantage, combined with inherent translation invariance through pooling, enables LeNet-5 to achieve superior accuracy on image classification tasks while requiring minimal domain expertise in feature design.

LeNet-5 originally employed average pooling and tanh activations, design choices that proved highly effective for the MNIST digit classification task.

# Chapter 2

# Theoretical Foundations

This chapter provides essential background on the key technologies relevant to neural network deployment on reconfigurable hardware: Field-Programmable Gate Arrays, Convolutional Neural Networks, and UART .

## 2.1 Field-Programmable Gate Arrays

A Field-Programmable Gate Array (FPGA) is a semiconductor device containing configurable logic blocks that can be programmed after manufacturing to implement custom digital circuits. Unlike Application-Specific Integrated Circuits (ASICs) with fixed functionality, or general-purpose processors and microcontrollers (CPUs, MCUs) that execute sequential software instructions, FPGAs offer a middle ground: hardware-level performance with post-fabrication reconfigurability.

FPGAs provide several key advantages for embedded applications. Computation is implemented through **spatial parallelism**—operations execute simultaneously across physically distributed logic rather than sequentially through shared arithmetic units. This yields **deterministic, low latency** without the overhead of caches or branch prediction. FPGAs also support **custom bit-width arithmetic** (e.g., 8-bit integers instead of 32-bit floats), and consume **power in the single-digit watt range** compared to 200+ watts for high-performance GPUs—making them particularly suitable for real-time embedded inference.


Figure 2.1: Digilent Basys3 Development Board featuring Xilinx Artix-7 XC7A35T FPGA

FPGAs are programmed using Hardware Description Languages (HDLs), the two dominant standards being **Verilog** and **VHDL**. These languages describe the logical structure of digital circuits at Register-Transfer Level (RTL) abstraction. An MCU fetches and executes a sequence of instructions stored in memory—the hardware is fixed, and behavior changes only by changing the program. An FPGA, by contrast, does not execute a program at all: the HDL description is synthesized and mapped onto the FPGA fabric during configuration, establishing a fixed network of logic gates, Look-Up Tables (LUTs), flip-flops, and Block RAM that directly implement the desired function. Computation occurs through signal propagation through this physical logic structure rather than sequential instruction dispatch. In this work, the entire inference engine is described in Verilog and compiled to a bitstream that configures the Artix-7 fabric at startup.

## 2.2 Convolutional Neural Networks

### 2.2.1 Artificial Neuron Model

Biological neurons receive electrical signals through *dendrites*, accumulate them in the *soma*, and fire an output pulse along the *axon* when the accumulated potential exceeds a threshold. Artificial neurons abstract this process mathematically. Given $n$ inputs $x_1, x_2, \ldots, x_n$ with learnable weights $w_1, w_2, \ldots, w_n$ and a scalar bias $b$, the neuron computes a weighted sum and applies a nonlinear *activation function* $f$:

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right) \tag{2.1}$$

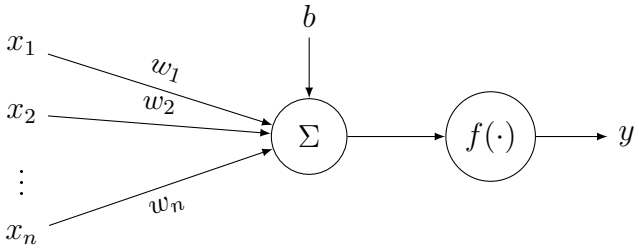



Figure 2.2: Artificial neuron: weighted inputs and a bias are summed, then transformed by activation $f$ to produce output $y$.

When many such neurons are arranged in **layers** and every neuron in layer $l$ connects to every neuron in layer $l-1$, the architecture is called a **Fully Connected** (FC), or *dense*, network. FC networks do not scale well to image data. A single $28 \times 28$ greyscale image already has 784 input values; a first hidden layer of 512 neurons requires $784 \times 512 = 401{,}408$ weights for that layer alone. For colour images at $224 \times 224 \times 3$ (e.g. the ImageNet benchmark), the input dimensionality rises to 150,528—yielding tens of millions of parameters per layer—while all spatial structure in the image is completely discarded. This parameter explosion, together with the inability to exploit local correlations, motivated the development of Convolutional Neural Networks.

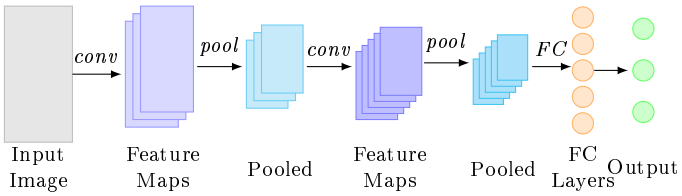



Figure 2.3: Conceptual CNN pipeline: stacked convolutional and pooling layers extract hierarchical spatial features; fully-connected layers map flattened features to class scores. See Figure 3.1 for the LeNet-5 variant used in this work.

Convolutional Neural Networks are deep learning architectures designed for processing grid-structured data, particularly images. The fundamental innovation is **weight sharing**: instead of connecting every input pixel to every output neuron (as in fully-connected networks), CNNs apply small learnable filters repeatedly across the input, dramatically reducing parameter counts while preserving spatial relationships.

Three core operations define CNNs: convolution for feature extraction, pooling for spatial reduction, and activation functions for nonlinearity. These operations are illustrated using LeNet-5 as an example (see Figure 3.1 in Section 3.1).

### 2.2.2 Convolution

Convolution slides learnable filters (typically 3×3 or 5×5 kernels) across the input, computing dot products at each position to detect local patterns like edges, textures, and

higher-level features. Each kernel acts as a feature detector, producing an activation map that highlights where specific patterns appear in the input image.

The LeNet-5 architecture (detailed in Figure 3.1, Section 3.1) processes 32×32 input images, but this implementation adapts it for native 28×28 MNIST images. To preserve spatial dimensions when applying 5×5 kernels, **padding** is employed—adding 2 pixels of zeros on each border effectively expands 28×28 to 32×32 before convolution. This allows the kernel to operate on edge pixels: when centered at the top-left corner (0,0) of the original image, the 5×5 kernel extends into the padded border, ensuring all original pixels receive equal receptive field coverage. Without padding, a 5×5 kernel on 28×28 input would produce only 24×24 output, losing spatial information at boundaries.

The **stride** parameter controls how many pixels the kernel moves between successive positions. A stride of 1 (used in LeNet-5 convolutional layers) produces dense feature maps, while larger strides reduce output dimensions and computational cost.

### 2.2.3 Pooling

Pooling operations downsample feature maps by applying reduction functions over local windows, providing two key benefits: spatial dimension reduction (decreasing computational requirements in subsequent layers) and limited translation invariance (making the network more robust to small input shifts).

Two common pooling strategies exist: **max pooling** selects the maximum value in each window (preserving the strongest activations), while **average pooling** computes the mean (providing smoother, more distributed representations). The LeNet-5 architecture (labeled as S2 and S4 in Figure 3.1) uses 2×2 average pooling with stride 2, which non-overlapping windows that halve both spatial dimensions. After the first convolutional layer produces 28×28 feature maps, average pooling reduces them to 14×14, cutting memory requirements and computation by 75% while retaining essential features.

### 2.2.4 Activation Functions

Activation functions introduce nonlinearity into neural networks, enabling them to learn complex decision boundaries. Without nonlinear activations, stacked layers collapse to a single linear transformation regardless of depth.

#### ReLU (Rectified Linear Unit)

ReLU is defined as $f(x) = \max(0, x)$ and has become the default activation in modern deep learning due to computational efficiency.

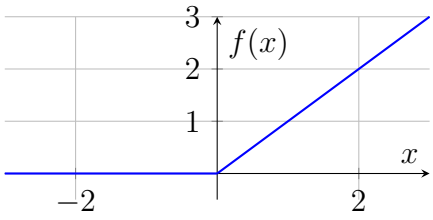



Figure 2.4: ReLU: $f(x) = \max(0, x)$

**Pros:** Computationally efficient, avoids vanishing gradients, sparse activation. **Cons:** "Dying ReLU" problem, not zero-centered.

### Tanh (Hyperbolic Tangent)

Tanh is defined as $f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$, producing an S-curve mapping to $[-1, 1]$.

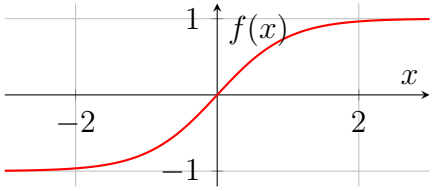



Figure 2.5: Tanh: $f(x) = \tanh(x)$

**Pros:** Zero-centered outputs, smooth gradients, stronger than sigmoid. **Cons:** Vanishing gradients for large $|x|$, computationally expensive. This LeNet-5 implementation uses tanh (matching the original 1998 design) via a 256-entry lookup table (Section 4.4).

### Sigmoid

Sigmoid is defined as $f(x) = \sigma(x) = \frac{1}{1+e^{-x}}$, mapping inputs to $(0, 1)$ for probabilistic interpretation.

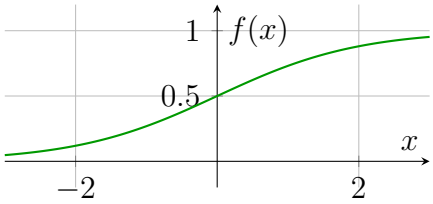



Figure 2.6: Sigmoid: $f(x) = \frac{1}{1+e^{-x}}$

**Pros:** Probabilistic interpretation, smooth and differentiable. **Cons:** Severe vanishing gradients, not zero-centered, computationally expensive. Now primarily used for binary classification outputs and LSTM gates.

### Comparison and FPGA Considerations

Modern CNNs predominantly use ReLU for hidden layers due to computational efficiency and superior gradient flow. For FPGA deployment, activation function choice significantly impacts hardware complexity: ReLU requires only simple thresholding logic, while tanh and sigmoid need lookup tables or exponential computations. This implementation uses tanh (matching the original 1998 LeNet-5 design) via a 256-entry lookup table (Section 4.4), achieving 97.6% accuracy on MNIST.

LeNet-5, developed by Yann LeCun in 1998 for handwritten digit recognition, established the canonical CNN architecture pattern (see detailed architecture diagram in Figure 3.1, Section 3.1): alternating convolutional/pooling layers (C1-S2-C3-S4) for hierarchical feature extraction, followed by fully-connected layers (C5-F6-OUTPUT) for classification. Despite its age, LeNet-5 remains pedagogically valuable due to its simplicity (61,706 parameters, 416,520 multiply-accumulate operations per inference) while demonstrating core principles applicable to modern architectures like ResNet and EfficientNet.

## 2.3 Universal Asynchronous Receiver-Transmitter

UART (Universal Asynchronous Receiver-Transmitter) is an asynchronous serial communication protocol widely used for point-to-point data transmission between microcontrollers, FPGAs, and host computers. Unlike synchronous protocols, UART requires no shared clock signal between transmitter and receiver. Instead, both devices agree on a baud rate (bits per second) in advance, and the receiver synchronizes on start bits marking each byte transmission. This simplicity makes UART ideal for FPGA-to-PC communication in embedded systems, including neural network weight loading and inference result transmission as implemented in this work.

For applications transmitting structured binary data (e.g., neural network weights), higher-level protocols use marker sequences to delimit packets. A key challenge is **binary-safe detection**: marker bytes may appear in arbitrary data payloads, requiring payload-length-aware parsing to prevent false packet termination.

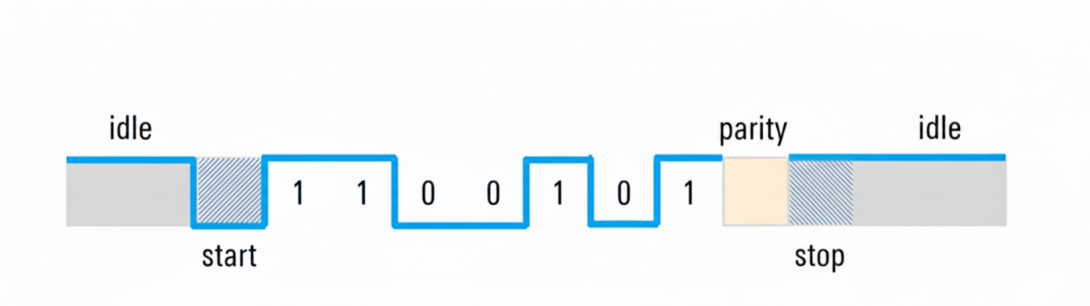



Figure 2.7: UART 8N1 Frame Format (Rohde & Schwarz 2020)

The standard UART frame format (8N1) consists of: 1 start bit (falling edge), 8 data

bits transmitted LSB-first, no parity, and 1 stop bit (return to idle high). The first falling edge from idle high to low marks the **start bit**, which triggers the receiver to synchronize and begin sampling data bits at the agreed baud rate. Following the start bit, 8 data bits are transmitted sequentially, least significant bit (LSB) first. The **parity bit** is an optional error-checking mechanism (P = even/odd parity, N = no parity); in 8N1 format, no parity bit is included. After the data bits, the **stop bit** returns the line to the idle high state, signaling the end of the frame.

Figure 2.7 illustrates the transmission of ASCII character 'S' (0x53 = 01010011 in binary). Due to LSB-first transmission, the bit sequence on the wire is reversed: 1-1-0-0-1-0-1-0. At 115,200 baud—a common rate for microcontroller/FPGA communication—each bit period is 8.68 microseconds, resulting in a complete 10-bit frame time of approximately 87 microseconds and maximum throughput of 11.5 KB/second.

# Chapter 3

# Training

This chapter describes the training pipeline for the LeNet-5 model, including the neural network architecture, dataset preparation, training process, and the critical quantization strategy that enables efficient FPGA implementation.

## 3.1 LeNet-5 Architecture

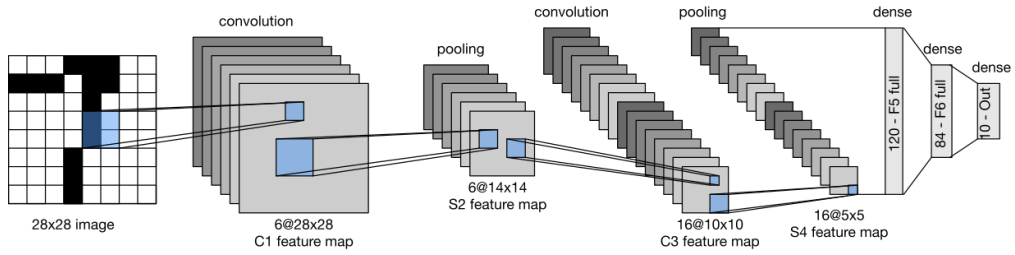



Figure 3.1: LeNet-5 Architecture (Wikimedia Commons 2026)

The implemented model is the classic LeNet-5 architecture (see Figure 3.1) optimized for FPGA deployment. The network consists of two convolutional layers with average pooling, followed by three fully-connected layers for classification (detailed specifications are provided in Section 4.3).

**Convolutional Layer 1:**

- 6 filters with $5 \times 5$ kernels,
- padding 2, stride 1,
- tanh activation,
- output: $6 \times 28 \times 28$,

- captures low-level features: edges, corners, curves.

**Average Pooling Layer 1:**

- $2 \times 2$ kernel, stride 2,
- output: $6 \times 14 \times 14$,
- subsampling for translation invariance.

**Convolutional Layer 2:**

- 16 filters with $5 \times 5$ kernels across 6 input channels,
- no padding, stride 1,
- tanh activation,
- output: $16 \times 10 \times 10$,
- combines features into higher-level patterns.

**Average Pooling Layer 2:**

- $2 \times 2$ kernel, stride 2,
- output: $16 \times 5 \times 5 = 400$ features,
- further spatial reduction.

**Fully Connected Layer 1:**

- 400 input features from flattened pool output,
- 120 output neurons,
- tanh activation.

**Fully Connected Layer 2:**

- 120 input features,
- 84 output neurons,
- tanh activation.

**Fully Connected Layer 3:**

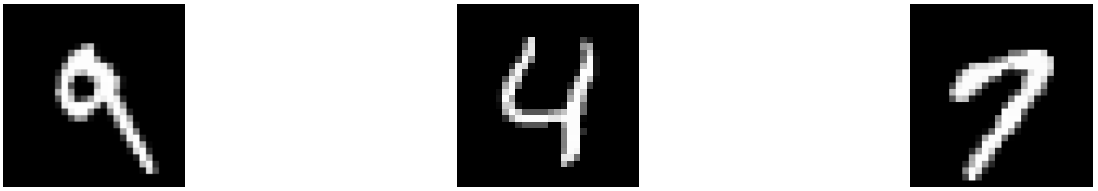
- 84 input features,
- 10 output neurons (one per digit class),
- no activation (raw logits for classification).

## 3.2 Dataset and Preprocessing

### 3.2.1 MNIST Dataset

The MNIST database of handwritten digits (LeCun, Cortes, and Burges 1998) contains:

- 60,000 training images,
- 10,000 test images,
- each image: $28 \times 28$ pixels, grayscale (0-255),
- 10 classes (digits 0-9).



(a) Digit 9 (b) Digit 4 (c) Digit 7

Figure 3.2: Example MNIST handwritten digits from the dataset (LeCun, Cortes, and Burges 1998)

The dataset split into training and testing sets serves a critical purpose in machine learning model development. The training set (60,000 images) is used exclusively for learning model parameters through backpropagation and gradient descent. The test set (10,000 images) provides an unbiased evaluation of the final model's performance on previously unseen data, ensuring that accuracy measurements reflect true generalization capability rather than memorization of training examples. This separation is essential for detecting overfitting, where a model performs well on training data but fails to generalize to new inputs. The MNIST train-test split is standardized across the machine learning community, enabling direct comparison of results with published literature and establishing reproducible benchmarks for handwritten digit classification systems.

### 3.2.2 Preprocessing Pipeline

The preprocessing transforms raw pixel values for neural network input:

$$x_{normalized} = \frac{x_{raw}/255 - \mu}{\sigma} \tag{3.1}$$

where $\mu = 0.1307$ and $\sigma = 0.3081$ are the MNIST dataset statistics.

For FPGA deployment, the normalized values are quantized to 8-bit signed integers:

$$x_{int8} = \text{clip}\left(\text{round}(x_{normalized} \times 127), -128, 127\right) \tag{3.2}$$

## 3.3 Training Configuration

The model is trained using PyTorch (Paszke et al. 2019) with the following hyperparameters:

Table 3.1: Training Hyperparameters

| Parameter | Value |
|---|---|
| Batch Size | 64 |
| Epochs | 10 |
| Learning Rate | 0.001 |
| Optimizer | Adam |
| Loss Function | Cross-Entropy |

**Batch Size** determines how many training samples are processed together before updating network weights—larger batches provide more stable gradient estimates but require more memory, while very small batches (e.g., 1) cause noisy updates that may prevent convergence, making the choice of 64 a practical balance between stability and computational efficiency. **Epochs** specify how many complete passes through the entire training dataset are performed—10 epochs means the network sees all 60,000 MNIST images 10 times. **Learning Rate** controls the step size during weight updates—smaller values (0.001) ensure stable convergence but slower training. **Optimizer** defines the weight update algorithm—Adam (Adaptive Moment Estimation) adapts learning rates per parameter and combines momentum for faster convergence. **Loss Function** measures prediction error—Cross-Entropy compares predicted class probabilities against true labels, penalizing confident incorrect predictions more heavily.

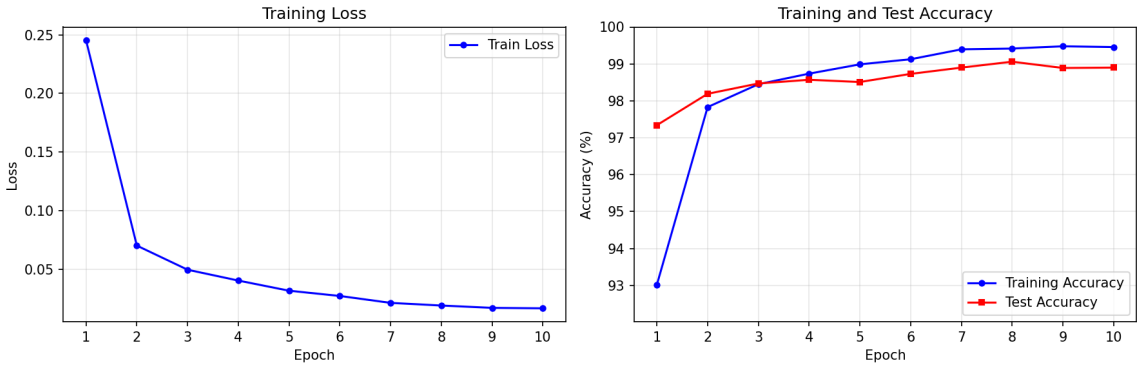



Figure 3.3: Training progress over 10 epochs showing loss convergence (left) and accuracy improvement (right). The model achieves 98.9% training accuracy and 98.8% test accuracy after 10 epochs.

The training curves demonstrate effective learning dynamics with rapid convergence in the initial epochs followed by gradual refinement. The loss decreases sharply from 0.25 to below 0.05 within the first two epochs, then continues gradual descent to approximately 0.02 by epoch 10. Training accuracy improves from approximately 93% to over 99%,

while test accuracy closely tracks at 98.8%, indicating excellent generalization capability. The close correspondence between training and test performance throughout all epochs demonstrates that the model avoids overfitting and successfully learns meaningful digit representations rather than memorizing training examples. This strong baseline performance at 98.9% accuracy provides an ideal starting point for the subsequent quantization process, which aims to preserve classification capability while enabling efficient fixed-point arithmetic on FPGA hardware.

## 3.4 Quantization Strategy

**Post-training static quantization** (PTQ) converts the trained floating-point model to fixed-point arithmetic without retraining. PTQ was chosen over quantization-aware training (QAT) because it requires no modification to the training pipeline and delivers sufficient accuracy for the MNIST task—the resulting model achieves approximately 1.3% lower test accuracy than the FP32 baseline. On the hardware side, INT8 multipliers consume far less LUT area and power than FP32 units, and the reduced memory footprint is essential for deployment on resource-constrained devices such as the Basys3.

Weights are quantized to 8-bit signed integers using a per-tensor scale factor:

$$W_{scale} = \frac{127.0}{\max(|W_{float}|)} \tag{3.3}$$

$$W_{int8} = \text{clip}\left(\text{round}(W_{float} \times W_{scale}),\, -128,\, 127\right) \tag{3.4}$$

Biases are kept in 32-bit integers to prevent accumulator overflow. The bias scale is propagated as $\text{InputScale} \times W_{\text{scale}}$; for layers that follow average pooling or tanh activation, the effective input scale is reduced accordingly (e.g. Conv2 and FC1 see $127/4 = 31.75$ instead of 127.0).

The FPGA inference pipeline processes each layer's output through four steps:

1. **Accumulate:** $acc = bias + \sum_i (pixel_i \times weight_i)$
2. **Arithmetic right shift:** $temp = acc \gg SHIFT$ (per-layer, see Table 3.2)
3. **Clip to INT8 range:** $temp = \text{clip}(temp,\, -128,\, 127)$
4. **Tanh LUT lookup:** $output = \text{tanh\_lut}[temp + 128]$

Table 3.2: Per-layer arithmetic right-shift values

| **Layer** | **SHIFT** | **Effective input scale** |
|---|---|---|
| Conv1 | 10 | 127.0 (normalized input) |
| Conv2 | 8 | 31.75 (post-pool, 127/4) |
| FC1 | 9 | 31.75 (post-pool) |
| FC2 | 10 | 127.0 (post-tanh) |

# Chapter 4

# Inference Implementation

This chapter describes the FPGA implementation of the CNN inference engine, including the hardware architecture, finite state machine design, memory organization, and communication protocols.

## 4.1 Target Hardware

The implementation targets the Digilent Basys3 development board (Digilent Inc. 2018), featuring:

Table 4.1: Target Hardware Specifications

| **Component** | **Specification** |
|---|---|
| FPGA Chip | Xilinx Artix-7 XC7A35T-1CPG236C |
| Logic Cells | 33,280 |
| Block RAM | 1,800 Kbit (225 KB) |
| Digital Signal Processing (DSP) slices | 90 |
| System Clock | 100 MHz |
| UART Baud Rate | 115,200 bps |

The Artix-7 family represents a cost-effective, low-power FPGA option suitable for educational and embedded applications, making it an appropriate platform for demonstrating neural network inference in resource-constrained environments.

## 4.2 System Architecture

The top-level design follows a modular architecture:

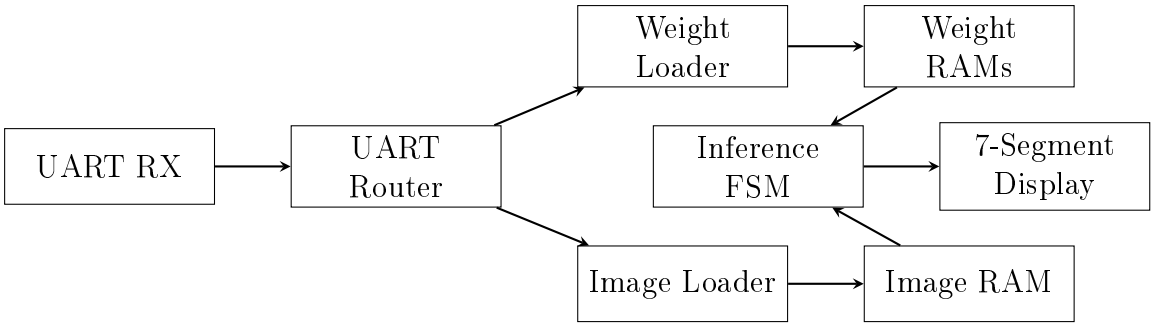



Figure 4.1: High-Level System Block Diagram

Figure 4.1 illustrates the complete FPGA inference pipeline, organized into three functional subsystems: the communication layer (UART RX, UART Router), the memory subsystem (Weight RAMs, Image RAM, working buffers), and the computation engine (Inference FSM). The modular architecture achieves clear separation of concerns, with each component fulfilling a specific role in the digit classification workflow. Data flows from the host PC through the UART interface, into dedicated memory structures, through the inference computation, and finally to the 7-segment display and optional result readback via UART TX.

The **communication layer** handles all data transfer between the host PC and FPGA. The UART RX module receives serial data at 115,200 baud using 8N1 format (8 data bits, no parity, 1 stop bit), which the UART Router then parses according to a binary-safe protocol using start/end marker sequences (0xAA 0x55 for weight packets, 0xBB 0x66 for image packets). The router implements payload-aware marker detection to prevent false end-marker detection within binary weight data, only validating end markers after receiving the complete expected payload (62,670 bytes for weights, 784 bytes for images). Based on the detected packet type, the router gates data to either the Weight Loader or Image Loader module, which then populate their respective memory structures.

The **memory hierarchy** stores all neural network parameters and working data required for inference. Weight RAMs store the complete set of 61,706 network parameters, including convolutional layer weights and biases (using distributed RAM for 0-cycle read latency), fully connected layer weights and biases (using Block RAM for higher density), and a 256-entry tanh lookup table for activation functions. Image RAM holds the $28\times28$ preprocessed input pixels (784 bytes total). The system also employs three working buffers (Buffer A, B, and C) implemented in distributed RAM to store intermediate activations between layers, enabling the FSM to access data with single-cycle latency during computation. Each buffer is strategically reused across multiple layers: Buffer A stores Conv1 output ($6\times28\times28$) then Conv2 output ($16\times10\times10$), Buffer B stores Pool1 output ($6\times14\times14$) then FC1 output (120 neurons), and Buffer C stores Pool2 output ($16\times5\times5$) then FC2 output (84 neurons). This reuse is possible because the FSM processes layers sequentially—once a subsequent layer consumes data from a buffer, that data is no longer

needed and the buffer can be overwritten. Multiple buffers are necessary because layers must simultaneously read input data while writing output results, preventing in-place updates to a single buffer.

The **Inference FSM** serves as the computational core, implementing a 44-state finite state machine that executes the complete LeNet-5 pipeline sequentially. The processing flow is: Conv1→Pool1→Conv2→Pool2→FC1→FC2→FC3. For each layer, the FSM orchestrates data fetching from appropriate memories, performs multiply-accumulate (MAC) operations using pure fixed-point integer arithmetic, applies tanh activation via lookup table where required, and stores results in working buffers. The complete inference executes 416,520 MAC operations per image, with the FSM managing different memory latencies (distributed RAM with 0-cycle latency versus Block RAM with 1-2 cycle latency) through carefully designed wait states. The final layer computes class scores for digits 0-9, with the FSM tracking the argmax to determine the predicted digit.

System operation follows a **two-phase protocol**. In Phase 1 (weight loading), the UART Router gates the Weight Loader to receive all 62,670 bytes of network parameters, then sets a `weights_loaded` flag to indicate readiness. In Phase 2 (inference), the system accepts image packets (784 bytes each), triggering the Inference FSM to execute the complete forward pass. Results are stored in dedicated output memories: Scores RAM (40 bytes for ten 32-bit class scores) and Predicted Digit RAM (1 byte). The predicted digit is immediately displayed on the 7-segment display, while the host PC can optionally request results via UART commands (0xCC for predicted digit, 0xCD for all class scores, 0xCE for cycle counter). This gating mechanism prevents out-of-sequence operations and ensures the inference engine has valid weights before attempting classification.

The modular design enables independent development and testing of each component. Detailed specifications for subsystems are provided in subsequent sections: Section 4.3 presents layer-by-layer computational requirements, Section 4.3 describes the Inference FSM state machine implementation, Section 4.4 explains the hybrid memory architecture and allocation strategy, and Section 4.5 specifies the complete UART communication protocol including binary packet formats.

## 4.3 MAC Operations

This section provides detailed computational requirements for each layer of the LeNet-5 architecture, including parameter counts and multiply-accumulate (MAC) operations per inference.

Table 4.2: LeNet-5 Layer Specifications

| Layer | Input Shape | Output Shape | Parameters | Operations |
|---|---|---|---|---|
| Input | – | $1 \times 28 \times 28$ | 0 | – |
| Conv1 | $1 \times 28 \times 28$ | $6 \times 28 \times 28$ | 156 | 117,600 MACs |
| AvgPool1 | $6 \times 28 \times 28$ | $6 \times 14 \times 14$ | 0 | – |
| Conv2 | $6 \times 14 \times 14$ | $16 \times 10 \times 10$ | 2,416 | 240,000 MACs |
| AvgPool2 | $16 \times 10 \times 10$ | $16 \times 5 \times 5$ | 0 | – |
| Flatten | $16 \times 5 \times 5$ | 400 | 0 | – |
| FC1 | 400 | 120 | 48,120 | 48,000 MACs |
| FC2 | 120 | 84 | 10,164 | 10,080 MACs |
| FC3 | 84 | 10 | 850 | 840 MACs |
| **Total** | | | **61,706** | **416,520 MACs** |

**Convolutional Layer 1:**

- *Parameters:* 6 output channels × (5×5 kernel × 1 input channel + 1 bias) = 6 × (25 + 1) = 156,
- *MAC Operations:* 6 output channels × 28×28 output positions × 5×5 kernel operations

$$\text{MACs}_{Conv1} = 6 \times 784 \times 25 = 117,600 \tag{4.1}$$

  each output pixel requires 25 multiplications (5×5 kernel) and 25 accumulations.

**Convolutional Layer 2:**

- *Parameters:* 16 output channels × (5×5 kernel × 6 input channels + 1 bias) = 16 × (150 + 1) = 2,416,
- *MAC Operations:* 16 output channels × 10×10 output positions × (5×5 kernel × 6 input channels)

$$\text{MACs}_{Conv2} = 16 \times 100 \times 150 = 240,000 \tag{4.2}$$

  each output pixel requires 150 multiplications (5×5×6 kernel volume) across all input channels.

**Fully Connected Layer 1:**

- *Parameters:* 400 inputs × 120 outputs + 120 biases = 48,000 + 120 = 48,120,
- *MAC Operations:* 120 output neurons × 400 input features

$$\text{MACs}_{FC1} = 120 \times 400 = 48,000 \tag{4.3}$$

  each neuron computes a dot product with all 400 inputs.

**Fully Connected Layer 2:**

- *Parameters:* 120 inputs × 84 outputs + 84 biases = 10,080 + 84 = 10,164,
- *MAC Operations:* 84 output neurons × 120 input features.

$$\text{MACs}_{FC2} = 84 \times 120 = 10,080 \tag{4.4}$$

**Fully Connected Layer 3 (Output):**

- *Parameters:* 84 inputs × 10 outputs + 10 biases = 840 + 10 = 850,
- *MAC Operations:* 10 output neurons × 84 input features

$$\text{MACs}_{FC3} = 10 \times 84 = 840 \tag{4.5}$$

this layer produces the final 10 class scores (one per digit 0-9).

**Total Computational Requirements:**

The complete network requires 416,520 MAC operations per inference, with convolutional layers dominating the computational workload (357,600 MACs, 86% of total). The relatively small parameter count (61,706) enables efficient storage in the Artix-7's Block RAM and distributed memory resources.

## 4.4 Inference FSM

As shown in Figure 4.1, the Inference FSM serves as the computational core of the system. The inference engine is implemented as a 44-state Finite State Machine for LeNet-5:

Table 4.3: Inference FSM States

| State | Name | Description |
|---|---|---|
| 0 | IDLE | Wait for start signal |
| 1 | L1_LOAD_BIAS | Load Conv1 bias from BRAM |
| 2 | L1_LOAD_BIAS_WAIT | Wait 1 cycle for BRAM read |
| 3 | L1_PREFETCH | Prefetch first pixel/weight |
| 4 | L1_CONV | 5×5 MAC operations |
| 5 | L1_TANH | Apply tanh activation via LUT |
| 6 | L1_SAVE | Write result to buffer |
| 7 | L1_POOL | Fetch 2×2 window values |
| 8 | L1_POOL_CALC | Sum and divide by 4 |
| 9 | L2_LOAD_BIAS | Load Conv2 bias |
| 10 | L2_LOAD_BIAS_WAIT | Wait for BRAM |
| 11 | L2_PREFETCH | Prefetch activation/weight |
| 12 | L2_CONV | 5×5×6 MAC operations |
| 13 | L2_TANH | Apply tanh activation via LUT |
| 14 | L2_SAVE | Write result to buffer |
| 15 | L2_POOL | Fetch 2×2 window values |
| 16 | L2_POOL_CALC | Sum and divide by 4 |
| 17 | FC1_LOAD_BIAS | Load FC1 bias |
| 18 | FC1_LOAD_BIAS_WAIT | Wait for BRAM |
| 19 | FC1_PREFETCH | Prefetch feature/weight |
| 20 | FC1_MULT | 400 MAC operations |
| 21 | FC1_TANH | Apply tanh activation via LUT |
| 22 | FC1_SAVE | Store result |
| 23 | FC2_LOAD_BIAS | Load FC2 bias |
| 24 | FC2_LOAD_BIAS_WAIT | Wait for BRAM |
| 25 | FC2_PREFETCH | Prefetch feature/weight |
| 26 | FC2_MULT | 120 MAC operations |
| 27 | FC2_TANH | Apply tanh activation via LUT |
| 28 | FC2_SAVE | Store result |
| 29 | FC3_LOAD_BIAS | Load FC3 bias |
| 30 | FC3_LOAD_BIAS_WAIT | Wait for BRAM |
| 31 | FC3_PREFETCH | Prefetch feature/weight |
| 32 | FC3_MULT | 84 MAC operations |
| 33 | FC3_NEXT | Store score, track argmax |
| 34 | DONE_STATE | Signal completion |
| 35 | FC1_MULT_WAIT | FC1 BRAM wait cycle 1 |
| 36 | FC2_MULT_WAIT | FC2 BRAM wait cycle 1 |
| 37 | FC3_MULT_WAIT | FC3 BRAM wait cycle 1 |
| 38 | FC1_MULT_WAIT2 | FC1 BRAM wait cycle 2 |
| 39 | FC2_MULT_WAIT2 | FC2 BRAM wait cycle 2 |
| 40 | FC3_MULT_WAIT2 | FC3 BRAM wait cycle 2 |
| 41 | FC1_PREFETCH2 | FC1 extra BRAM prefetch wait |
| 42 | FC2_PREFETCH2 | FC2 extra BRAM prefetch wait |
| 43 | FC3_PREFETCH2 | FC3 extra BRAM prefetch wait |

### 4.4.1 FSM Operation Overview

The inference state machine orchestrates the complete forward pass through the LeNet-5 network, processing one input image to produce classification scores for all 10 digit classes. The FSM architecture comprises 44 states organized in two groups: states 0-34 form the main sequential processing pipeline, while states 35-43 provide additional BRAM latency compensation states (MULT_WAIT, MULT_WAIT2, and PREFETCH2 for each fully-connected layer). The FSM begins in the IDLE state and transitions through each layer sequentially upon receiving a start signal. The complete inference pipeline consists of alternating convolutional and pooling operations followed by fully-connected classification layers, with each layer performing load-compute-activate-store cycles before advancing to the next layer.

At a high level, the FSM executes five distinct processing phases: (1) Conv1 with average pooling reduces the 28×28 input to 6 feature maps at 14×14 resolution, (2) Conv2 with average pooling further processes these to 16 feature maps at 5×5 resolution, (3-5) three fully-connected layers progressively reduce the 400-dimensional flattened feature vector to 120, then 84, and finally 10 class scores. Each convolutional and fully-connected layer follows an identical state pattern: load bias from memory, prefetch operands to hide memory latency, perform multiply-accumulate operations, apply tanh activation via lookup table, and store results to working buffers. The additional BRAM-specific states (35-43) handle the 1-2 cycle read latency inherent to Block RAM, ensuring correct data availability during fully-connected layer weight fetching.

### 4.4.2 Detailed State Machine Operation

Each layer's computation follows a structured pipeline designed to maximize throughput while accounting for memory access latencies. The state machine implements explicit wait states (LOAD_BIAS_WAIT, PREFETCH2, MULT_WAIT) to handle Block RAM's registered output delay—critical for the fully-connected layers where weight storage in BRAM introduces 1-2 cycle read latency compared to the zero-latency distributed RAM used for convolutional weights.

**Convolutional Layer Pipeline:** Figure 4.2 illustrates the complete state machine for convolutional layers.

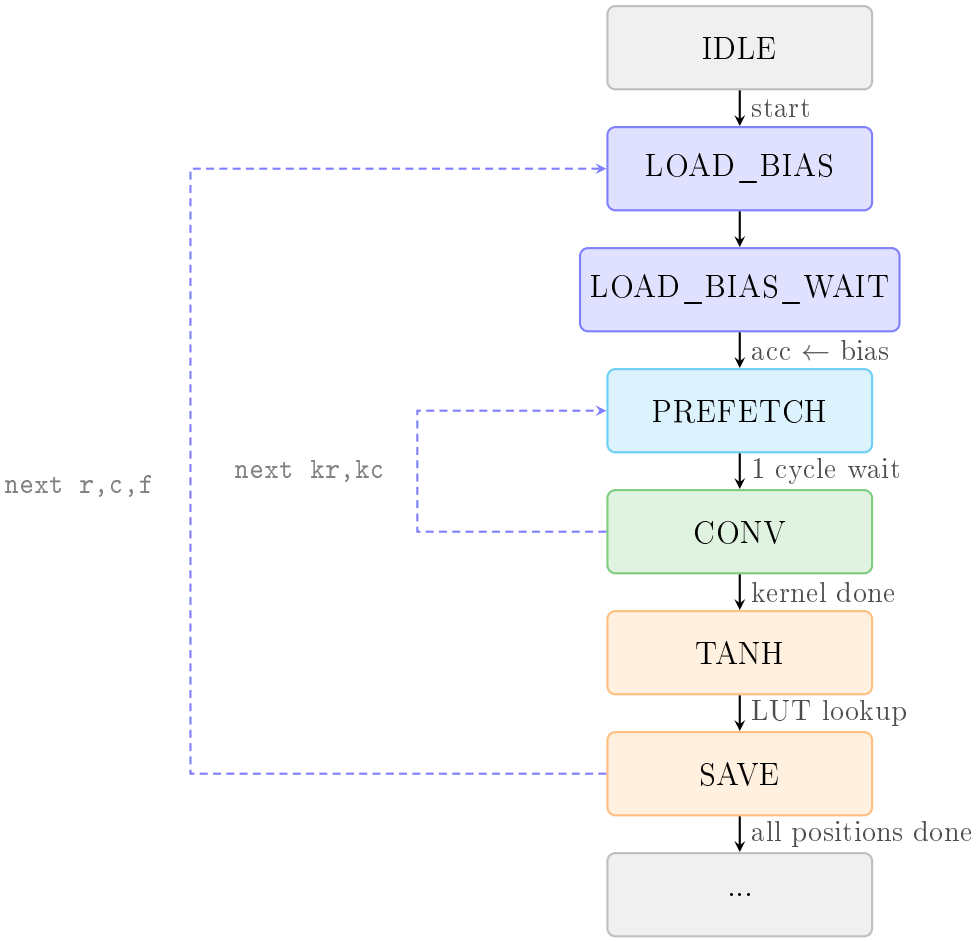



Figure 4.2: Convolutional layer FSM pattern

The convolutional layer pipeline implements a sequential state machine that processes each output position through multiple stages. Each of the two convolutional layers (L1, L2) instantiates this pattern with layer-specific state names. The FSM begins in the IDLE state, waiting for a start signal to commence processing. Upon activation, the state machine transitions to LOAD_BIAS, which requests the layer's bias value from memory. The LOAD_BIAS_WAIT state follows, introducing a one-cycle delay to accommodate Block RAM read latency and initializing the accumulator with the bias value (acc ← bias, as shown on the transition label). The PREFETCH state prepares the first pixel and weight operands for the convolution operation with a "1 cycle wait" as indicated. The CONV state (green) forms the computational core, executing kernel×kernel×channels multiply-accumulate operations—Layer 1 performs 25 MACs per position (5×5 kernel, 1 input channel), while Layer 2 performs 150 MACs per position (5×5 kernel, 6 input channels). A dashed loop arrow labeled "next kr,kc" (next kernel row, kernel column) indicates that the CONV state iterates over all kernel positions, returning to PREFETCH for each successive weight-pixel pair. After completing all kernel multiplications (indicated by the "kernel done" transition label), the FSM advances to the TANH state (orange), which clips the accumulated result to 8-bit range (-128 to 127), computes the lookup table

index by adding a 128 offset, and retrieves the activated value from the 256-entry tanh LUT (labeled "LUT lookup"). The SAVE state (orange) writes this activated result to the appropriate working buffer (Buffer A for Conv1 output, Buffer A again for Conv2 output) at the computed spatial address. A second dashed loop labeled "next r,c,f" (next row, column, filter) returns execution from SAVE back to LOAD_BIAS, iterating over all output feature map positions and filters until the entire convolutional layer output is generated. Following complete convolution of all feature maps (indicated by "all positions done"), the FSM transitions to the ellipsis state (...) representing continuation to the next pipeline stage (pooling). Solid arrows represent forward transitions through the pipeline, while dashed arrows indicate iteration loops at different granularities—the inner "next kr,kc" loop iterates over individual kernel elements, while the outer "next r,c,f" loop iterates over all spatial positions and output channels.

**Average Pooling Pipeline:** Figure 4.3 shows the detailed architecture of the average pooling pipeline.

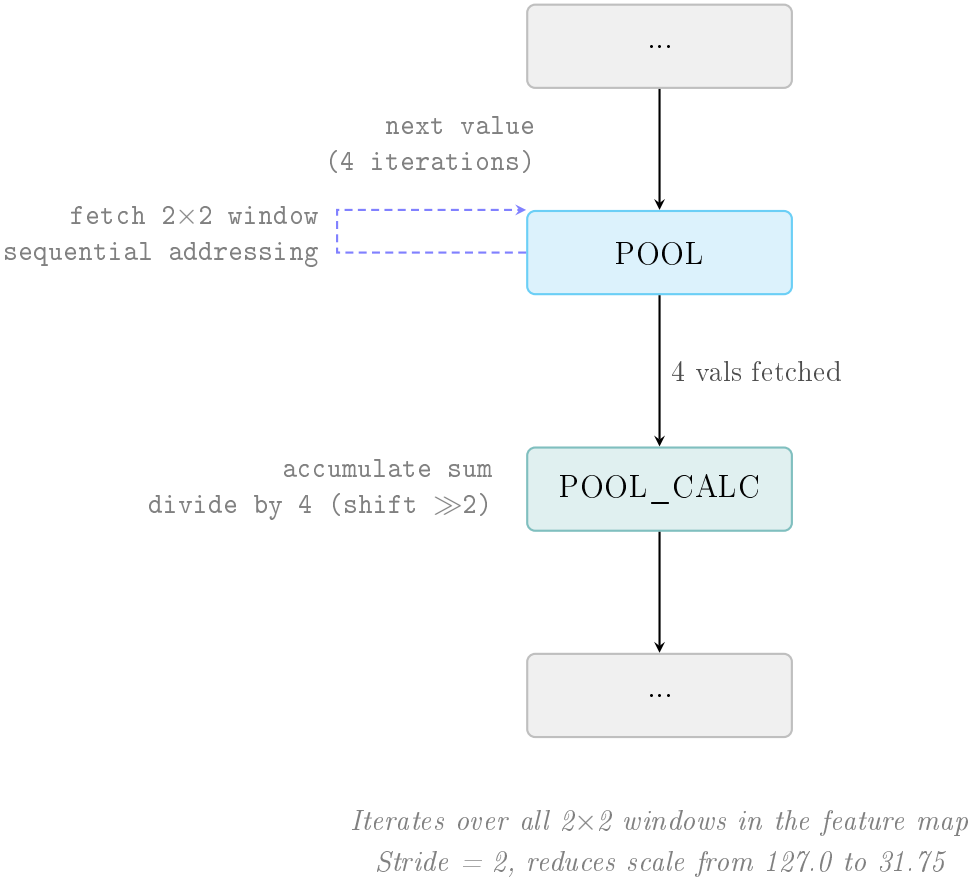



Figure 4.3: Average pooling pipeline architecture

The pooling pipeline implements a 5-cycle operation for each 2×2 window, executing after convolutional layers complete their feature map generation. The diagram shows the core two-state pooling operation embedded within the larger inference pipeline, with ellipsis states (...) at the beginning and end indicating continuation from the previous

convolution stage and transition to the next processing stage. The POOL state (cyan block) handles the iterative fetching and initial accumulation of the four values in a 2×2 window. This state executes 4 times per window, as indicated by the dashed self-loop arrow labeled "next value (4 iterations)" that returns from POOL back to itself. During each iteration, the state performs sequential addressing to read one value (v[0], then v[1], then v[2], then v[3]) from the previous layer's output buffer stored in distributed RAM. The left-side annotation describes the operations: "fetch 2×2 window / sequential addressing". After all 4 values have been fetched and accumulated (indicated by the solid arrow labeled "4 vals fetched"), the FSM transitions to the POOL_CALC state (teal block). The POOL_CALC state performs two operations as described in the left annotation: "accumulate sum / divide by 4 (shift $\gg$2)". The accumulation completes the summation begun in POOL, and the division is implemented through an arithmetic right shift by 2 bits, which efficiently computes integer division by 4. This critical operation reduces the activation scale from 127.0 (the maximum value range entering pooling) to 31.75 (the maximum range after division), affecting quantization parameters in subsequent layers. The averaged result is then written to the next working buffer (Buffer B after Pool1, Buffer C after Pool2) at the appropriate spatial address. The FSM then transitions to the trailing ellipsis (...) representing continuation to the next pipeline stage. Below the states, an italicized note explains that this entire sequence "Iterates over all 2×2 windows in the feature map" with "Stride = 2" (non-overlapping windows), reducing spatial dimensions by half—for example, transforming 28×28 feature maps to 14×14. The note emphasizes the scale reduction from 127.0 to 31.75, a crucial detail for maintaining numerical precision throughout the quantized network.

**Fully-Connected Layer Pipeline:** Figure 4.4 depicts the fully-connected layer state machine with BRAM latency compensation states highlighted.

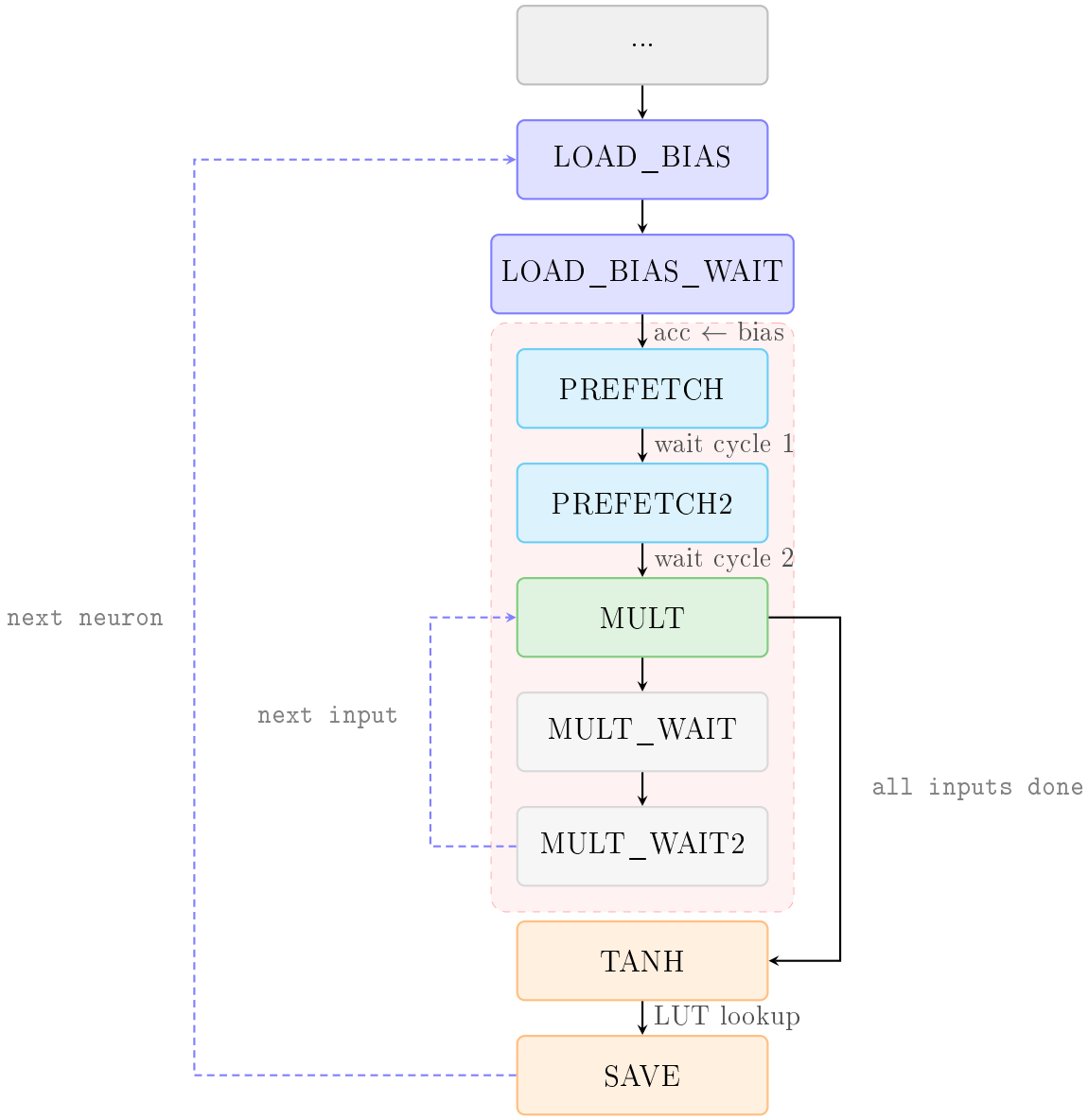



Figure 4.4: Fully-connected layer FSM pattern

Dense layers exhibit more complex state sequences due to Block RAM latency, requiring explicit wait states to compensate for the 1–2 cycle read delay inherent in BRAM-based weight storage. Each of the three fully-connected layers (FC1, FC2, FC3) instantiates this pattern with layer-specific state names. The diagram shows all ten states involved in computing a single neuron output, with a shaded red region highlighting the four BRAM latency compensation states. The FSM begins at the top with the "..." (ellipsis) state representing the previous layer's completion. Execution proceeds to LOAD_BIAS, which requests the neuron's bias value from Block RAM. The LOAD_BIAS_WAIT state follows, introducing a one-cycle delay to accommodate BRAM read latency and initializing the accumulator with the bias value (acc ← bias, as labeled on the transition arrow). The

PREFETCH state issues a read request for the first weight from BRAM. Because BRAM has registered outputs, the data is not immediately available—thus the PREFETCH2 state (cyan, part of the shaded latency compensation region) waits one additional cycle (labeled "wait cycle 1") before the weight becomes valid. After the second wait cycle ("wait cycle 2"), the FSM transitions to the MULT state (green), which forms the computational core. In MULT, the FSM issues a read request for the next weight and simultaneously accumulates the multiply-accumulate product using the weight that became available from the previous request. The MULT_WAIT state (gray, part of the shaded region) provides the first wait cycle for the newly-requested weight data. The MULT_WAIT2 state (gray, part of the shaded region) provides the second wait cycle. A dashed loop arrow labeled "next input" returns execution from MULT_WAIT2 back to MULT, iterating over all input features—FC1 processes 400 inputs, FC2 processes 120 inputs, and FC3 processes 84 inputs. This alternating pattern of request-wait-accumulate continues until all input features have been multiplied and accumulated. When the final input is processed (indicated by "all inputs done" on the right-side arrow), the FSM exits the multiplication loop and transitions to the TANH state (orange). The TANH state clips the accumulated result to 8-bit range (-128 to 127), computes the lookup table index by adding a 128 offset, and retrieves the activated value from the 256-entry tanh LUT (indicated by the "LUT lookup" label). The SAVE state (orange) writes this activated neuron output to the appropriate working buffer (Buffer B for FC1, Buffer C for FC2). A final dashed loop arrow labeled "next neuron" returns execution from SAVE back to LOAD_BIAS, iterating over all output neurons in the layer—FC1 produces 120 neurons, FC2 produces 84 neurons, and FC3 produces 10 class scores. The italicized note at the bottom clarifies that "FC3 omits TANH/SAVE; uses FC3_NEXT instead"—the final output layer skips activation and instead uses a specialized FC3_NEXT state that maintains a running argmax tracker to determine the predicted digit class. Solid arrows represent forward sequential transitions, while dashed arrows indicate iteration loops at different granularities (inner loop over inputs, outer loop over neurons).

**Output Layer:** The final FC3 layer (states 33-37) differs slightly by omitting activation—raw logits are stored directly to the scores RAM. The FC3_NEXT state maintains a running argmax tracker, comparing each class score and updating the predicted digit index. When all 10 scores are computed, the FSM transitions to DONE_STATE, asserting the completion signal and returning to IDLE to await the next inference request.

### 4.4.3 Convolution Implementation

Each convolutional output requires multiple clock cycles:

**Layer 1 (per output position):**

- 1 cycle: load bias,

- 1 cycle: wait for BRAM,
- 1 cycle: prefetch,
- 25 cycles: 5×5 MAC operations,
- 1 cycle: save result,
- total: ∼30 cycles per position × 4,704 positions = ∼141,120 cycles.

**Layer 2 (per output position):**

- 150 cycles: 5×5×6 MAC operations,
- total: ∼155 cycles per position × 1,600 positions = ∼248,000 cycles.

### 4.4.4 Average Pooling Implementation

The average pooling operation uses a 5-step pipeline per 2×2 window:

1. Set address for value 0, initialize accumulator
2. Read value 0, add to accumulator, set address for value 1
3. Read value 1, add to accumulator, set address for value 2
4. Read value 2, add to accumulator, set address for value 3
5. Read value 3, compute average (sum ≫ 2), write result

## 4.5 Memory Architecture

### 4.5.1 Hybrid Memory Strategy

The memory hierarchy shown in Figure 4.1 consists of Weight RAMs, Image RAM, and working buffers. The implementation employs a **hybrid memory architecture** combining distributed RAM (LUT-based) and Block RAM (BRAM) to balance performance and resource utilization:

**Distributed RAM:** Used for frequently-accessed working memories requiring single-cycle read latency:

Listing 4.1: Distributed RAM Template

```
(* ram_style = "distributed" *)
reg [7:0] ram [0:SIZE-1];

// Combinational read (0-cycle latency)
assign rd_data = ram[rd_addr];

// Synchronous write
always @(posedge clk) begin
    if (wr_en) ram[wr_addr] <= wr_data;
end
```

This eliminates pipeline bubbles that would occur with Block RAM's registered outputs, critical for intermediate activation buffers accessed every clock cycle during convolution operations.

**Block RAM:** Used for larger, less frequently accessed memories such as neural network weights. Block RAM provides high-density storage (16 blocks, 32% utilization) with 1-2 cycle read latency, which is acceptable for weight fetching as the FSM can absorb this delay in multi-cycle MAC operations. This allocation strategy conserves LUT resources for logic and distributed buffers.

### 4.5.2 Memory Organization

Table 4.4: Memory Allocation

| Memory | Size (bytes) | Type | Purpose |
|---|---|---|---|
| Conv Weights RAM | 2,550 | Distributed | Conv1+Conv2 weights |
| Conv Biases RAM | 88 | Distributed | 22 × 32-bit biases |
| FC Weights RAM | 58,920 | Block RAM | FC1+FC2+FC3 weights |
| FC Biases RAM | 856 | Distributed | 214 × 32-bit biases |
| Tanh LUT | 256 | Distributed | 256-entry activation table |
| Buffer A | 4,704 | Distributed | Conv1 output (6×28×28) |
| Buffer B | 1,176 | Distributed | Pool1 output (6×14×14) |
| Buffer C | 400 | Distributed | Pool2/FC2 output |
| Image RAM | 784 | Distributed | Input image |
| Scores RAM | 40 | Distributed | Output scores |
| **Total** | **∼68 KB** | | |

## 4.6 Communication Protocol

### 4.6.1 UART Router

The UART Router shown in Figure 4.1 serves as the central coordinator for all incoming UART data. The system employs a dedicated **UART Router** module to manage signal multiplexing between the UART receiver and multiple consumer modules (weight loader, image loader, command handlers). The router's primary functions are:

- **Packet Detection:** recognize protocol marker sequences (two-byte start/end markers),
- **Signal Routing:** route decoded payloads to appropriate destination modules (weight_loader or image_loader) based on detected marker type,
- **Flow Control:** forward data bytes only when destination modules are ready, maintaining protocol synchronization.

The router uses payload-aware marker detection to ensure robustness: end markers are validated only after the complete payload (62,670 bytes for weights, 784 bytes for image) has been received. This prevents false transitions triggered by marker-like byte sequences in binary data.

Operation is gated by a `weights_loaded` status signal that enforces protocol ordering: weight loading is allowed only when `weights_loaded = 0`, while image loading and command processing are allowed only when `weights_loaded = 1`. This prevents out-of-sequence operations and ensures the inference engine has valid weights before attempting inference.

Command bytes (0xCC for predicted digit, 0xCD for all class scores, 0xCE for cycle counter) are processed immediately in the idle state and forwarded to dedicated command readers for result transmission via UART TX. The 0xCE command returns the latched cycle count measured by a hardware counter integrated into the inference FSM; the counter increments every clock cycle from the start of inference until the FSM reaches `DONE_STATE`, at which point the value is frozen and held until queried. This allows the host PC to retrieve the exact number of clock cycles consumed by a single inference pass without relying on software-side timing.

### 4.6.2 UART Configuration

- baud rate: 115,200 bps,
- frame format: 8N1 (8 data bits, no parity, 1 stop bit),
- clock frequency: 100 MHz.

### 4.6.3 Protocol Markers

Table 4.5: Protocol Markers

| Marker | Bytes | Direction | Purpose |
|---|---|---|---|
| `0xAA 0x55` | Start | PC → FPGA | Begin weight transfer |
| `0x55 0xAA` | End | PC → FPGA | End weight transfer |
| `0xBB 0x66` | Start | PC → FPGA | Begin image transfer |
| `0x66 0xBB` | End | PC → FPGA | End image transfer |
| `0xCC` | Command | PC → FPGA | Request predicted digit |
| `0xCD` | Command | PC → FPGA | Request all scores |
| `0xCE` | Command | PC → FPGA | Request cycle counter |

### 4.6.4 Binary Safety

The protocol uses payload-length-aware parsing to prevent false end-marker detection in binary data:

Listing 4.2: Binary-Safe Marker Detection

```
// Only check end markers AFTER receiving expected payload
if (byte_count >= WEIGHT_SIZE &&
    prev_byte == WEIGHT_END1 &&
    rx_data == WEIGHT_END2) begin
    state <= DONE_W;
end
```

### 4.6.5 Flow Control

Python scripts use chunked transmission to prevent UART buffer overflow:

Listing 4.3: Chunked Transmission

```
def send_chunked(ser, data, chunk_size=32, delay=0.010):
    for i in range(0, len(data), chunk_size):
        chunk = data[i:i+chunk_size]
        ser.write(chunk)
        ser.flush()
        time.sleep(delay)  # 10ms delay per chunk
```

# Chapter 5

# Verification

The testing and verification process relies on a three-step methodology:

1. **Reference Logit Generation:** Creating a reliable Python simulation that accounts for quantized weights, fixed-point arithmetic with layer-specific shift values, and the tanh lookup table identical to the FPGA memory. This simulation serves as the source of reference logits for all subsequent verification steps.

2. **Isolated Inference Verification:** Testing bit-exact correctness of the `inference` module in the `tb_inference` testbench. This step bypasses the UART communication stack and directly verifies that the RTL produces identical outputs to the Python reference.

3. **Physical FPGA Verification:** End-to-end testing on actual hardware using the `compare_fpga_vs_python` script. Images are sent via UART, and the returned logits are compared against the Python reference to confirm correct operation of the complete system including communication and control logic.

This methodology was validated on 9,500 MNIST test images, achieving 100% bit-exact match across all 10 class scores with zero discrepancies.

## 5.1 Python Bit-Exact Simulation

The Python simulation replicates every step of the FPGA arithmetic pipeline:

Listing 5.1: Bit-Exact Convolution Simulation

```
def convolve_layer(input_vol, weights, biases, shift, tanh_lut):
    # For each output position, replicate FPGA datapath:
    acc = np.sum(window * kernel)                # MAC (like DSP48)
    acc += bias_val                              # Add bias
    acc = acc >> shift                           # Arithmetic right
       shift
    acc = tanh_lut[np.clip(acc, -128, 127) + 128]  # Clip to 8-bit,
        offset to [0,255], lookup tanh activation from 256-entry
       precomputed table
    return acc
```

Key aspects of bit-exact simulation:

- sequential MAC operations using Python integers to replicate FPGA fixed-point arithmetic,
- arithmetic right shift (») for fixed-point scaling with layer-specific shift values,
- clipping to 8-bit signed range before tanh LUT access,
- tanh activation via direct table lookup, identical to FPGA memory access pattern.

## 5.2 Verilog Testbench

The `tb_inference` testbench isolates and tests the `inference` module independently, bypassing the system-level UART stack. Test vectors are generated offline from a Python reference implementation that performs bit-exact arithmetic replication of the FPGA's quantized pipeline. For each test image, the Python model computes all 10 class logits and these results are stored in hexadecimal memory files alongside input pixel data and quantized weights.

The testbench loads 16 consecutive images from the MNIST dataset. For each image, it loads the pixel data and weights from memory files, executes the FPGA inference module, and compares the computed class scores against the Python-generated golden reference scores. The console output displays the per-image logit comparison, the FPGA-predicted digit, and the ground-truth label, enabling quick verification that the FPGA inference matches the Python reference across the test set.

## 5.3 End-to-End FPGA Test

The `compare_fpga_vs_python` script automates end-to-end testing on physical hardware:

1. Load and preprocess MNIST image
2. Run Python simulation to get expected scores
3. Send image to FPGA via UART
4. Wait for inference completion
5. Request scores via `0xCD` command
6. Compare bit-exact match and accuracy

# Chapter 6

# Results and Comparison

This chapter presents the experimental results and comparison with simpler models and related work.

## 6.1 Classification Accuracy

All tests were conducted on 9,500 images from the MNIST test set. Testing yields:

Table 6.1: Classification Accuracy Results

| **Implementation** | **Accuracy** |
|---|---|
| PyTorch (FP32) | 98.9% |
| Python Simulation (INT8) | 97.6% |
| FPGA Hardware | 97.6% |

The 1.3% accuracy drop from quantization is acceptable for practical embedded applications.

## 6.2 Resource Utilization

Table 6.2: FPGA Resource Utilization (Artix-7 XC7A35T)

| **Resource** | **Used** | **Available** | **Utilization** |
|---|---|---|---|
| LUTs (Logic) | 1,650 | 20,800 | 8% |
| LUTs (RAM) | 2,930 | 20,800 | 14% |
| Flip-Flops | 1,468 | 41,600 | 4% |
| Block RAM | 16 | 50 (36Kb) | 32% |
| DSP Slices | 3 | 90 | 3% |

Resource utilization values are reported from the Vivado post-synthesis summary.

The implementation targets the Basys3 development board (shown in Figure 2.1) featuring the Xilinx Artix-7 XC7A35T FPGA. Notable observations:

- balanced resource allocation: 8% LUTs for logic, 14% for distributed RAM,
- Block RAM used for weight storage (16 blocks, 32% utilization),
- minimal DSP usage (3 slices, 3.3%) - most arithmetic in LUT-based logic,
- low flip-flop utilization (4%) indicates efficient combinational design,
- total LUT utilization: 22% (4,582 / 20,800), leaving room for expansion.

## 6.3 Inference latency

Table 6.3: Inference Timing Breakdown

| **Stage** | **Operations** | **Est. Cycles** |
|---|---|---|
| Conv1 | 117,600 MACs | ∼141,000 |
| AvgPool1 | 1,176 windows | ∼7,000 |
| Conv2 | 240,000 MACs | ∼248,000 |
| AvgPool2 | 400 windows | ∼2,400 |
| FC1 | 48,000 MACs | ∼145,000 |
| FC2 | 10,080 MACs | ∼31,000 |
| FC3 | 840 MACs | ∼2,500 |
| **Total** | 416,520 MACs | ∼577,000 cycles |

FC layer cycle counts include 2-cycle BRAM read latency wait states (MULT_WAIT, MULT_WAIT2) required for weight fetching from Block RAM.

To verify these theoretical estimates, a hardware cycle counter was integrated into the inference FSM. The counter resets when inference begins (transition from IDLE state) and increments each clock cycle until completion (DONE_STATE). The final count is available via a dedicated UART command (0xCE), allowing the host PC to query exact cycle counts for performance analysis.

Table 6.4: Measured vs. Theoretical Inference Timing

| **Metric** | **Theoretical** | **Measured** |
|---|---|---|
| Cycles per inference | ∼577,000 | 576,182 |
| Latency @ 100 MHz | ∼5.8 ms | 5.762 ms |
| Throughput | ∼172 img/s | ∼174 img/s |

The measured cycle count of 576,182 confirms the theoretical estimate within 0.14% accuracy. Deterministic execution is a key advantage of FPGA implementation over software-based inference, where cache behavior and OS scheduling introduce timing variability.

## 6.4 Comparison with Simpler Models

Prior to implementing the full LeNet-5 architecture, two simpler models with structurally related designs were developed and evaluated on the same MNIST dataset. These preliminary models—a Softmax Regression classifier and a two-layer Multi-Layer Perceptron—served as architectural stepping stones, establishing the training and evaluation pipeline before introducing convolutional layers.

Table 6.5: Comparison with Simpler Architectures

| **Model** | **Parameters** | **Accuracy** | **Memory** |
|---|---|---|---|
| Softmax Regression | 7,850 | ∼92% | ∼8 KB |
| 2-Layer Multi-Layer Perceptron (MLP, 784-128-10) | 101,770 | ∼97% | ∼100 KB |
| LeNet-5 | 61,706 | 97.6% | ∼68 KB |

Key observations:

- LeNet-5 achieves highest accuracy (97.6%) despite moderate memory footprint,
- convolutional weight sharing reduces parameters compared to equivalent MLP (61K vs. 101K),
- hierarchical feature extraction through multiple convolutional layers provides significant accuracy boost over simple models.

This implementation achieves competitive accuracy among comparable FPGA implementations while maintaining moderate resource requirements.

## 6.5 Key Results

The implemented LeNet-5 network achieves:

- **97.6% accuracy** on MNIST classification (INT8 quantized),
- **bit-exact** correspondence between Python and FPGA,
- **∼5.8 ms inference latency** at 100 MHz clock frequency,
- **∼68 KB** total memory footprint.

These results demonstrate the viability of deploying quantized CNNs on resource-constrained FPGAs for embedded inference applications.

# Chapter 7

# Summary

This chapter provides a comprehensive walkthrough of the complete LeNet-5 FPGA implementation project, summarizing the journey from initial model training through hardware deployment and verification. A detailed system architecture diagram is provided in Appendix A.

## 7.1 Project Overview

This thesis presents an end-to-end implementation of the LeNet-5 convolutional neural network for MNIST handwritten digit classification on a resource-constrained FPGA platform. The project demonstrates the complete workflow required to deploy neural networks on embedded hardware: training with quantization-aware considerations, RTL implementation with careful memory architecture design, and rigorous verification ensuring bit-exact correspondence between software and hardware.

The system architecture (see Appendix A, Figure A.1) illustrates the complete data flow from the host PC through UART communication to the FPGA inference engine. The design follows a modular approach with clear separation between communication, memory, and computation subsystems.

## 7.2 Training and Quantization Pipeline

The project began with training a LeNet-5 model using PyTorch on the MNIST dataset of 60,000 handwritten digit images. The floating-point model achieved 98.9% classification accuracy. Post-training static quantization converted all weights and activations to 8-bit integers, enabling efficient fixed-point arithmetic on FPGA fabric. The quantization pipeline required careful attention to scale propagation through layers, particularly after average pooling operations that reduce activation magnitudes by a factor of 4.

The quantized model preserves 97.6% accuracy—a modest 1.3% drop that demonstrates the robustness of INT8 quantization for this classification task. All 61,706 network parameters (weights and biases) were exported in a binary format suitable for UART transmission to the FPGA.

## 7.3 Hardware Implementation

The FPGA implementation targets the Digilent Basys3 board featuring the Xilinx Artix-7 XC7A35T chip. The inference engine is implemented as a 44-state finite state machine that executes the complete LeNet-5 pipeline: two convolutional layers with average pooling, followed by three fully-connected classification layers.

Key architectural decisions include:

- hybrid memory architecture combining distributed RAM (for zero-latency activation buffers) with Block RAM (for high-density weight storage),
- explicit BRAM latency handling through dedicated wait states in the FSM,
- strategic buffer reuse across layers to minimize memory footprint,
- 256-entry tanh lookup table for efficient activation function implementation.

The communication subsystem implements a binary-safe UART protocol with payload-aware marker detection, preventing false packet termination when marker-like sequences appear in weight data. A two-phase protocol ensures weights are loaded before inference begins.

## 7.4 Verification and Results

Verification followed a three-tier methodology: Python bit-exact simulation replicating FPGA arithmetic, isolated RTL testbench verification, and end-to-end hardware testing on the physical FPGA. This approach was validated on 9,500 MNIST test images, achieving 100% bit-exact match between Python simulation and FPGA hardware across all 10 class scores.

The implementation achieves:

- 97.6% classification accuracy (INT8 quantized),
- 5.762 ms inference latency at 100 MHz (576,182 clock cycles),
- approximately 174 images per second throughput,
- 68 KB total memory footprint,
- 22% LUT utilization, 32% Block RAM utilization, 3% DSP utilization.

## 7.5 Conclusions

This project demonstrates that deploying quantized convolutional neural networks on resource-constrained FPGAs is both feasible and practical. The sequential FSM-based architecture prioritizes simplicity and low resource utilization over raw throughput, making it suitable for embedded applications where power efficiency and deterministic latency are critical requirements.

The complete system—from PyTorch training through quantization, RTL implementation, and hardware verification—provides a reference implementation for FPGA-based neural network inference. The modular design enables independent development and testing of subsystems, while the comprehensive verification methodology ensures reliable operation.

The significant unused FPGA resources (particularly DSP slices at 3% utilization) indicate substantial headroom for future optimization through parallel architectures, as discussed in the following chapter.

# Chapter 8

# Future Development of the Project

The current implementation utilizes only 3 of 90 available DSP slices (3%), leaving significant computational resources unused. A natural evolution would transition from the sequential FSM-based architecture to a parallel DSP array approach for dramatic throughput improvements.

The Artix-7 XC7A35T provides 90 DSP48E1 slices optimized for multiply-accumulate operations. Instead of computing one MAC per cycle sequentially, a parallel architecture would instantiate multiple DSP blocks to process convolution operations simultaneously. For example, unrolling the 5×5 kernel computation across 25 DSP slices reduces kernel computation from 25 cycles to 1 cycle. Processing 3 filters in parallel (75 DSPs total) could achieve approximately 50-75× speedup in convolutional layers.

However, this requires solving the **memory bandwidth bottleneck**: parallel computation demands 50+ simultaneous memory reads per cycle, while standard FPGA memory provides only 1-2 read ports per block. Solutions include line buffers (shift registers caching pixel rows for parallel access), weight replication across multiple BRAM blocks, and memory banking with careful address mapping to avoid port conflicts.

Table 8.1: Sequential vs. Parallel Architecture Comparison

| **Metric** | **Sequential (Current)** | **Parallel (75 DSPs)** |
|---|---:|---:|
| DSP Utilization | 3 (3%) | 75 (83%) |
| Latency @ 100MHz | 5.8 ms | ∼150 $\mu$s |
| Throughput | 172 img/s | ∼6,600 img/s |
| Control Complexity | Simple FSM | Parallel dataflow |

Table 8.1 compares the sequential FSM implementation with a hypothetical parallel DSP array architecture. The cycle counts for the sequential implementation are detailed in Table 6.3. The parallel architecture assumes 75 DSP blocks (83% of the 90 available on Artix-7), allocating 3 filters × 25 DSP slices each for full kernel parallelization in Conv1. This leaves 15 DSP blocks unused, as processing all 6 Conv1 filters simultaneously would require 150 DSPs, exceeding available resources.

Parallel architectures offer ∼40× latency reduction but require near-complete DSP uti-

lization, complex memory subsystems, sophisticated dataflow control, and significantly higher power consumption. The sequential approach is optimal for resource-constrained deployments prioritizing simplicity and low power, while parallel designs suit applications demanding ultra-low latency and high throughput with sufficient FPGA resources available.

**Touch Panel Integration:** A compelling extension would integrate a resistive or capacitive touch panel overlay, enabling users to draw digits directly on the FPGA board for real-time classification. This requires implementing a touch controller interface (typically Serial Peripheral Interface (SPI) or Inter-Integrated Circuit (I2C)), a drawing canvas with gesture recognition, and downsampling logic to convert touch coordinates to the 28×28 pixel format expected by the neural network. Such integration would transform the system from a laboratory demonstration into an interactive embedded application.

**Systolic Array Architecture:** Beyond simple DSP unrolling, a systolic array represents a more sophisticated approach to parallel matrix multiplication. In a systolic design, processing elements (PEs) are arranged in a 2D grid where data flows rhythmically through the array—weights remain stationary while activations stream through horizontally and partial sums accumulate vertically. This architecture maximizes weight reuse (each weight is used by multiple PEs before being replaced), dramatically reducing memory bandwidth requirements compared to naive parallelization. Systolic arrays form the computational core of modern neural network accelerators including Google's TPU, making this a natural evolution path for FPGA-based inference engines.

# Bibliography


Digilent Inc. (2018). *Basys3 FPGA Board Reference Manual*. URL: https://digilent.com/reference/programmable-logic/basys-3/reference-manual (visited on 02/11/2026).

Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton (2012). "ImageNet classification with deep convolutional neural networks". In: *Advances in Neural Information Processing Systems*. Vol. 25. Lake Tahoe, NV: Curran Associates.

LeCun, Yann, Léon Bottou, Yoshua Bengio, and Patrick Haffner (1998). "Gradient-based learning applied to document recognition". In: *Proceedings of the IEEE* 86.11.

LeCun, Yann, Corinna Cortes, and Christopher J. Burges (1998). *The MNIST database of handwritten digits*. URL: https://yann.lecun.org/exdb/mnist/ (visited on 01/30/2026).

Paszke, Adam et al. (2019). "PyTorch: An imperative style, high-performance deep learning library". In: *Advances in Neural Information Processing Systems*. Vol. 32. Vancouver, BC, Canada: Curran Associates.

Rohde & Schwarz (2020). *UART: A Hardware Communication Protocol Understanding UART*. Video tutorial. URL: https://www.youtube.com/watch?v=sTHckUyxwp8 (visited on 02/11/2026).

Venieris, Stylianos I. and Christos-Savvas Bouganis (2016). "fpgaConvNet: A framework for mapping convolutional neural networks on FPGAs". In: *IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM)*. Washington, DC: IEEE.

Wikimedia Commons (2026). *LeNet-5 architecture diagram*. URL: https://en.wikipedia.org/wiki/LeNet#/media/File:LeNet-5_architecture.svg (visited on 01/30/2026).

# Appendix A

# System Architecture Diagram

Figure A.1 provides a detailed view of the complete FPGA inference system, showing the data flow from host PC through UART communication to the inference engine and output display. This comprehensive diagram illustrates the interconnection between all major subsystems: the communication layer (UART RX/TX, protocol router), memory hierarchy (weight storage, image buffer, working memories), computation engine (inference FSM), and output interfaces (7-segment display, UART result readback).

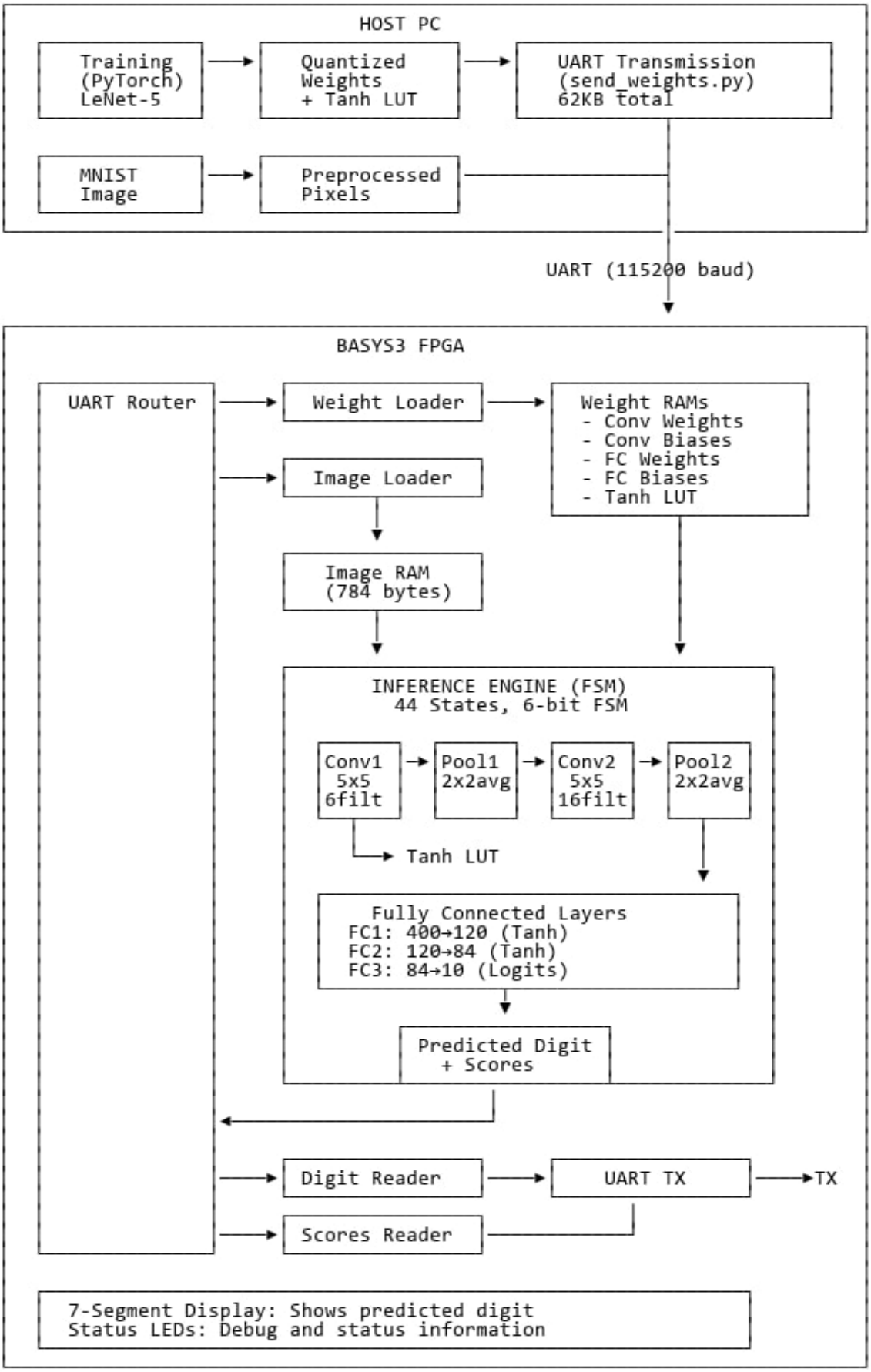



Figure A.1: Complete System Architecture: Host PC to FPGA Inference Pipeline